

Gerência Adaptativa e Programável de Qualidade de Serviço em Redes IP

Miguel F. de Castro^{1*}, Dominique Gaïti²,
Abdallah M'hamed¹, Rossana M. C. Andrade³

¹RST - Institut National des Télécommunications
9 rue Charles Fourier - 91011 - Evry Cedex - France

²LM2S - Université de Technologie de Troyes
12 rue Marie Curie - 10010 - Troyes Cedex - France

³DC - Universidade Federal do Ceará
Campus do Pici - Bloco 910 - 60.455-220 - Fortaleza/CE - Brasil

Resumo. A Internet da próxima geração deverá lidar com restrições impostas pela convergência de serviços e pela necessidade de Qualidade de Serviço (QoS). Este artigo propõe uma gerência adaptativa da QoS em redes IP baseada na tecnologia das Redes Ativas e Programáveis. Neste novo paradigma de gerência, meta-comportamentos podem ser personalizados e instalados em entidades de redes, como roteadores. Esses meta-comportamentos são expressos em uma nova linguagem chamada CLAM (Compact Language for Adaptive Management). Uma arquitetura para suportar esse tipo de gerência é também apresentada e utilizada em um estudo de caso sobre gerência adaptativa de filas de espera em roteadores.

Abstract. New generation Internet is coming with several new constraints imposed by service convergence and Quality of Service (QoS) capabilities. This article proposes an adaptive IP QoS management based on Active and Programmable Networks in which customized meta-behaviors can be configured on network elements. These meta-behaviors are expressed in a new language called CLAM (Compact Language for Adaptive Management). A support architecture enhanced with these active and programmable capabilities is presented and used in a case study on adaptive queue management in routers.

1. Introdução

As aplicações multimídia estão convergindo para uma única infra-estrutura onde os usuários podem ter acesso integrado a todos estas aplicações e também desejar pagar mais para ter um serviço de melhor qualidade do que os demais usuários. Para viabilizar este novo ambiente, uma nova arquitetura baseada na classificação e no tratamento diferenciado das diversas classes de tráfego (*DiffServ*) foi proposta em [Blake et al., 1998]. Nesta arquitetura, cada uma das classes de serviços pode ter

*Financiado pela Fundação CAPES. E-mail: miguel.castro@int-evry.fr.

sua própria prioridade e requisitos. Não obstante, a filosofia do melhor esforço persiste como uma classe de serviço para usuários que não têm nenhuma necessidade especial em termos de QoS (Qualidade de Serviço).

A coexistência de diversas classes de serviço e a necessidade de suprir diferentes requisitos exigidos pelos serviços multimídia transformaram a tarefa de gerenciamento de redes que, por exemplo, deverá agora lidar com as diversas soluções propostas para resolver o problema de QoS. Para cada uma dessas soluções possíveis, pesquisadores encontraram pontos fortes e fracos. Portanto, como as condições de tráfego variam constantemente em volume e natureza, escolher uma boa configuração de QoS não é fácil [Iannaccone et al., 2001]. Além do desempenho dos serviços, diversos outros parâmetros têm que ser levados em conta para essa escolha, como maximização da vazão, justiça (*fairness*) na alocação dos recursos excedentes e custo de implementação (processamento e/ou armazenamento).

Uma boa solução para esse problema pode ser a inclusão de um certo nível de inteligência no interior da rede. Esta possibilidade já vem mostrando evidências de bons benefícios à tarefa de gerência de redes [de Castro et al., 2002, Sterbenz, 2002]. A tecnologia das redes ativas e programáveis pode ser usada para levar essa inteligência para o interior da rede.

O objetivo desse trabalho é utilizar a tecnologia das redes ativas e programáveis para assegurar uma gerência de QoS eficiente e adaptativa às redes IP. A idéia por trás dessa gerência adaptativa é a de habilitar elementos de rede com recursos de inteligência suficientes para permitir o monitoramento e a readaptação da gerência de QoS de acordo com as condições de tráfego atuais. Para isso, este artigo apresenta uma nova arquitetura baseada nas redes ativas e programáveis para realizar a gerência adaptativa e programável de redes IP, mais notadamente no domínio do suporte a QoS. O objetivo dessa nova arquitetura é tornar a gerência de QoS mais dinâmica, flexível e autônoma. Associada a esta arquitetura foi desenvolvida uma nova linguagem de programação simplificada chamada CLAM (*Compact Language for Adaptive Management*). Este trabalho também apresenta um estudo de caso onde a arquitetura proposta é aplicada à gerência adaptativa de escolha de mecanismos de gerenciamento de fila de espera com o objetivo de melhorar o desempenho dos serviços.

Este artigo está organizado da seguinte forma. A Seção 2 apresenta uma breve descrição da tarefa de gerenciamento de QoS em redes IP. Na Seção 3 é abordada a definição da gerência adaptativa e trabalhos correlatos são citados. A nova arquitetura de gerência adaptativa baseada em redes ativas é descrita na Seção 4. Para ilustrar o uso dessa arquitetura, um estudo de caso baseado na gerência de fila de espera em roteadores é apresentado na Seção 5. A Seção 6 apresenta e comenta os resultados obtidos, enquanto a Seção 7 expõe as conclusões e trabalhos futuros.

2. O Gerenciamento de QoS em Redes IP

A Internet da próxima geração (NGI - *Next Generation Internet*) acomodará a convergência dos serviços de rede. Como as versões anteriores do IP não oferecem *a priori* suporte a QoS, novas propostas foram apresentadas para resolver esse problema. O suporte de novas mídias na Internet não é uma tarefa fácil. A variedade de tipos de informação e os potenciais problemas de interoperabilidade entre essas mídias (*e.g.* coexistência de conexões TCP e fluxos UDP, que geralmente são grandes consumidores de largura de banda e não possuem mecanismos de controle de fluxo) [Hong et al., 2001] criam novas responsabilidades para a gerência de redes. O suporte a QoS deve lidar com diversos tipos de aplicações, fluxos e usuários, cada um potencialmente com seus próprios requisitos e prioridades.

Com o objetivo de solucionar o problema de QoS nas redes Internet, a IETF (*Internet Engineering Task Force*) propôs duas novas arquiteturas: Serviços Integrados (IntServ - *Integrated Services*) [Braden et al., 1994] e Serviços Diferenciados (DiffServ - *Differentiated Services*) [Blake et al., 1998].

A arquitetura IntServ é baseada na reserva de recursos por fluxo. Apesar do IntServ oferecer possibilidade de assegurar QoS, a necessidade de uma granularidade de controle por fluxo fez da escalabilidade a sua principal falha. Entretanto, as idéias, conceitos e mecanismos desenvolvidos para o IntServ foram reaproveitados em outros trabalhos sobre QoS. Por exemplo, o serviço de carga controlada (*Controlled Load Service*) influenciou o desenvolvimento da arquitetura DiffServ e um esquema similar de reserva de recursos foi incorporado ao MPLS (*Multi-Protocol Label Switching*) para oferecer garantias de largura de banda ao tráfego tronco em *backbones*.

Como mencionado anteriormente, através do estudo das restrições do IntServ, pesquisadores desenvolveram uma outra arquitetura de suporte a QoS mais escalável e flexível, baseada na diferenciação dos serviços em classes, chamada DiffServ. A operação do DiffServ é baseada em PHBs (*Per-Hop Behaviors*). PHBs são implementados em nós interiores e de borda de uma rede através de algoritmos de gerência de filas e de escalonamento [Blake et al., 1998]. Entretanto, existem várias implementações possíveis para um PHB, uma vez que o único aspecto definido é o comportamento de encaminhamento (*forwarding*) observável externamente para um PHB e não os mecanismos que o implementam.

A implementação de um PHB envolve a adoção de um conjunto de mecanismos de gerência de filas de espera e de escalonamento. A escolha desses mecanismos, que não é prevista na especificação da arquitetura, é um aspecto crucial para uma gerência de QoS efetiva. Além disso, existem evidências de que mecanismos de gerência de QoS sejam mais aplicáveis a determinadas condições de tráfego do que em outras. Como o número de mecanismos disponíveis é bastante grande, é difícil escolher uma só configuração que resulte em bom desempenho para as aplicações em diversas situações de tráfego ao mesmo tempo em que o custo de implementação (processamento e memória) seja racionalizado.

3. A Gerência Adaptativa de QoS

Escolher uma boa configuração de gerência de QoS entre todos os mecanismos disponíveis é uma tarefa bastante complexa [Iannaccone et al., 2001]. Esta escolha deve levar em conta não só o desempenho das aplicações, mas também a maximização do uso da largura de banda, a distribuição justa dos recursos e o custo de implementação. O desafio é encontrar uma configuração que leve em conta todos esses fatores. Por exemplo, tomando como principal parâmetro o custo de implementação, o mecanismo de escalonamento WFQ (*Weighted Fair Queuing*) traz bom desempenho para a maioria das situações de tráfego. Entretanto, podem existir situações de tráfego mais simplificadas onde o emprego do WFQ não seja justificado por causa de sua complexidade em detrimento a mecanismos de escalonamento mais simplificados como o tratamento FIFO (*First-In First-Out*).

Alguns trabalhos mostram que é possível mecanismos mais complexos de gerência de QoS apresentarem pouca – ou mesmo nenhuma – melhora de desempenho quando comparado com mecanismos mais simples, dependendo do cenário de tráfego. Por exemplo, Christiansen *et al.* (2001) provaram que o mecanismo RED (*Random Early Detection*) não proporciona ganho de desempenho quando comparado ao FIFO com *Drop-Tail* para enlaces carregando primordialmente tráfego *Web*. Portanto, o descarte prematuro de pacotes imposto por esse mecanismo assim como a complexidade de sua implementação (que inclui geração de número randômicos) não são justificados nessa situação de tráfego. Por outro lado, existem provas de que o FIFO com *Drop-Tail* mantém altas médias de taxa de ocupação das filas, além de propiciar fenômenos como sincronização global de fluxos (*global synchronization*) [Floyd and Jacobson, 1993]. Tal cenário mostra a necessidade de uma rede que adapte o seu comportamento ao “ambiente”, *i.e.* a eventos e restrições que ocorrem em torno dos elementos de rede. Neste trabalho, introduz-se o termo “gerência comportamental da QoS” para designar esse tipo de gerência, onde as entidades de rede são parcialmente responsáveis por sua operação e manutenção.

A gerência de redes adaptativa tem sido discutida na comunidade científica. Embora seja possível encontrar abordagens onde a adaptação ocorre no núcleo da rede, a maioria das propostas explora o gerenciamento adaptativo baseado na operação fim-a-fim, mais notadamente no controle de fluxo em serviços multímídia [Furini and Towsley, 2001].

As tecnologias ativas, que abrangem as redes ativas e programáveis e os Agentes Móveis [Hu and Chen, 2000], possuem as características propícias para resolver o problema do gerenciamento comportamental de QoS. Esses dois domínios têm sido estudados como soluções possíveis para problemas de gerenciamento de redes. As redes ativas podem ser usadas para dar flexibilidade à criação e lançamento de serviços, enquanto os agentes móveis são comumente usados para distribuir inteligência e poder de processamento dentro da rede. Essas duas abordagens são bastante similares, mas cada uma traz seus benefícios para a gerência de redes. Enquanto

os agentes móveis são geralmente aplicados para a coleta de informações e atuam no nível de aplicação, as redes ativas são mais adequadas para operações de gerência em camadas mais baixas do modelo OSI. Alguns trabalhos relacionados com o gerenciamento adaptativo através de redes ativas são apresentados a seguir.

Schwartz *et al.* (2000) propuseram uma arquitetura de gerenciamento baseada em redes ativas denominada “*Smart Packets*”. Essa abordagem é baseada no transporte de código compacto através da rede e da execução desses códigos em nós remotos com o objetivo de realizar tarefas de gerenciamento de redes, tais como SNMP *GETs* e *SETs*. O projeto *Smart Packets* não prevê a instalação de código persistente dentro dos nós. Portanto, esta característica impossibilita a criação de códigos que criam os comportamentos dentro de cada nó.

Jackson *et al.* (2002) propuseram o SENCOMM (*Smart Environment for Network Control, Monitoring and Management*), uma outra arquitetura de gerenciamento baseada em redes ativas. Exatamente como a abordagem “*Smart Packet*”, SENCOMM é baseada na execução remota de código compacto, porém sem instruções persistentes.

As abordagens apresentadas acima exploram as tecnologias ativas com o objetivo de realizar um gerenciamento de redes flexível e eficiente. Entretanto, tais abordagens não são capazes de suportar a programabilidade de *comportamentos* personalizados nos nós, uma vez que o emprego de código persistente é necessário para manter a operação de monitoramento e a conseqüente adaptação dos comportamentos. Portanto, uma abordagem mais completa para a gerência de redes baseada nas tecnologias ativas, foco da proposta desse artigo, faz-se necessária.

4. Uma Arquitetura para a Gerência Adaptativa e Programável de QoS

Este artigo propõe uma nova arquitetura onde um plano de gerência é adicionado à arquitetura em camadas da Internet. Esta arquitetura de camadas em três dimensões é inspirada no modelo de referência B-ISDN (*Broadband Integrated Services Digital Network*) [ITU-T, 1992], que tem um plano separado onde todas as funções de gerência são executadas. Entretanto, o objetivo específico dessa nova arquitetura é oferecer o suporte à gerência adaptativa de QoS em redes IP através da exploração da tecnologia das redes ativas e programáveis. A Figura 1 ilustra essa nova proposta.

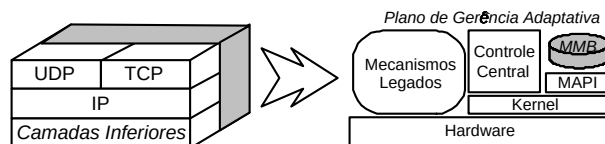


Figura 1: Arquitetura Funcional da Gerência Adaptativa.

O novo plano de gerência adaptativa é constituído por seis componentes apresentados na Figura 1 e detalhados a seguir. O hardware do sistema é compartilhado

entre duas entidades: o conjunto de Mecanismos Legados e o *Kernel* de Gerência. No topo do *Kernel*, há a Base de Módulos de Gerência (MMB - *Management Base Modules*), a Interface de Programação de Aplicação de Gerência (MAPI - *Management Application Programming Interface*) e o Controle Central.

Essas entidades reunidas integram o Ambiente de Execução (*Execution Environment*) de gerência adaptativa. As instâncias do Ambiente de Execução são chamadas *meta-configurações* e são definidas através da implementação de *módulos programáveis de gerência adaptativa*. Uma meta-configuração define o comportamento do nó através do relacionamento entre esses módulos programáveis. Exemplos de tipos de módulos programáveis são: estatística, política, notificação, entre outros. A relação entre esses módulos é ilustrada na Figura 2.



Figura 2: Procedimento de Gerência Adaptativa.

Dentre os demais componentes da arquitetura estão os Mecanismos Legados, que são os mecanismos internos intrínsecos à arquitetura de rede escolhida. Os exemplos são os mecanismos de gerência de fila de espera (*e.g.*, *Drop-Tail*, ECN e RED) e de escalonamento (*e.g.*, WFQ). Estes mecanismos foram escolhidos para serem armazenados separadamente dos mecanismos personalizados porque são estritamente padronizados e inerentes à arquitetura de redes empregada (*e.g.*, IPv6, IPv4, etc.), e assim podem se beneficiar de uma implementação sobre *hardware*, por exemplo.

O *Kernel* de Gerência é um sistema operacional responsável pela gestão das instâncias do Ambiente de Execução de Gerência Adaptativa, sendo responsável pelo controle dos módulos de gerência que são construídos no topo do MAPI e armazenados no MMB. O *Kernel* também pode controlar recursos locais como filas, memória e o poder de processamento.

O MMB é uma memória persistente que armazena as implementações dos módulos de gerência personalizados. As atualizações do MMB são completamente controladas pelo Controle Central e, conseqüentemente, por entidades como o centro de gerência do domínio.

A MAPI, por sua vez, contém as funções e os procedimentos necessários para que os projetistas de gerência tenham acesso às funções e aos recursos de baixo nível da entidade de rede, como filas e escalonamento de pacotes, entre outros.

O papel do *Controle Central* é gerenciar a operação de instâncias do Ambiente de Execução, monitorar o *status* da rede e os eventos inerentes, e decidir, baseado nas políticas definidas no meta-comportamento, a ativação, desativação e modificação dos módulos de gerência adaptativa disponíveis. O Controle Central também é responsável por servir informação personalizada de gerência ao centro de gerência do domínio.

A segurança nesta arquitetura é garantida pela proteção de entrada de pacotes especiais de gerência encapsulados através do protocolo ANEP (*Active Network Encapsulation Protocol*) no interior da rede [ANEP, 2004], *i.e.* os roteadores de borda devem inexoravelmente filtrar pacotes deste tipo, exceto se provenientes de uma interface confiável (porta de acesso do centro de gerência da provedora, por exemplo). Além disso, o emprego de uma sintaxe limitada na linguagem de programação protege a arquitetura de ameaças como *loops* infinitos. Apesar de simples, essa medida de segurança tem se mostrado bastante eficaz, sendo bastante empregada no domínio das DSL's (*Domain-Specific Languages*).

Como a arquitetura suporta a abstração de domínios de operadora (*carrier domains*), o fator de escalabilidade não representa um problema. Além disso, o número de funções de gerência ativa injetadas na rede não tem relação direta com o número de usuários ou de fluxos. Em relação a desempenho, esta arquitetura foi baseada no conceito ForCES (*Forwarding and Control Elements Separation*) [Khosravi et al., 2003], onde as funções e infraestruturas de gerência e de encaminhamento de pacotes são distintas e colaborativas, *i.e.* os planos da arquitetura não apresentam interdependência. Por exemplo, no processo de coleta de estatísticas baseadas em informações de camadas mais altas em pacotes, a análise é realizada sobre cópias de pacotes interceptados na interface. Assim, a função de encaminhamento de pacotes não é bloqueada enquanto a análise é feita.

4.1. CLAM: Uma Nova Linguagem para a Gerência Adaptativa

Os *meta-comportamentos* são expressos nos elementos de rede em uma nova linguagem chamada CLAM (*Compact Language for Adaptive Management*). Esta linguagem é projetada para compor todos os tipos de módulos programáveis previstos na arquitetura. Por motivos de segurança, a sintaxe desta linguagem é bastante limitada, não permitindo, por exemplo, estruturas de laço e ponteiros de memória.

Os códigos em CLAM são encapsulados em pacotes ANEP (*Active Network Encapsulation Protocol*) [ANEP, 2004], que por sua vez são encapsulados diretamente sobre o IP. O protocolo ANEP especifica mecanismos de encapsulamento de pacotes ativos para transporte sobre diferentes tipos de mídia. Os formatos sugeridos na especificação ANEP suportam o transporte sobre infraestruturas existentes (como IPv4 ou IPv6) ou mesmo transporte direto sobre a camada de enlace. Mecanismos simplificados inerentes de verificação de integridade de código asseguram a boa recepção dos módulos programáveis nos elementos de rede. A fragmentação de código CLAM em diferentes pacotes também é possível através da utilização do campo "OPTIONS" do pacote ANEP.

Programas CLAM utilizam funções e procedimentos definidos na MAPI, incluindo acesso a informações locais no estilo MIB e chamadas a procedimentos de gerência, como ativação e desativação de mecanismos de gerência, mudança de parâmetros desses mecanismos, entre outros.

Como diferentes módulos programáveis podem co-existir no âmbito do ambiente de execução CLAM, um mecanismo simples de detecção e resolução de conflitos baseado em prioridades de regras foi adotado. Como um módulo programável pode depender de vários outros módulos (*e.g.*, uma política depende de várias estatísticas), um mecanismo de verificação de dependências controla a ativação dos módulos programáveis. Um exemplo de código CLAM é apresentado a seguir:

```
define statistic udp_load prior 0 clock 0.001 as
  collectPacket(pkt);
  proto_id = getTranspProtId(pkt);
  vectorInsert(counter, proto_id);
end
```

O código apresentado acima define uma estatística *udp_load*, com prioridade 0, desencadeada a cada 1 *ms*, definindo que uma amostra de pacote deve ser coletada e analisada. O identificador do protocolo em nível de transporte desse pacote será armazenado em um vetor (persistente) chamado *counter*. Outros módulos programáveis poderão fazer uso desta informação. Por exemplo, é possível se obter uma estimativa da proporção de tráfego UDP sobre o tráfego agregado através da contagem de identificadores do protocolo UDP no vetor *counter*.

5. Estudo de Caso: Uma Gerência Adaptativa de Filas de Espera

Vários trabalhos têm sido apresentados na literatura propondo novas abordagens para mecanismos de gerência de fila de espera, como o RED, RIO, WRED, BLUE, etc. Essas novas propostas vêm responder a uma falha ou omissão de um algoritmo anterior em tratar uma determinada situação de tráfego, por exemplo. Christiansen *et al.* mostram que o RED (*Random Early Detection*) [Braden et al., 1998] não apresenta ganhos significativos quando comparado ao FIFO com *Drop-Tail* quando o enlace carrega primordialmente tráfego *Web*. Iannaccone *et al.* (2001) vão mais além e afirmam que o FIFO com *Drop-Tail* ainda é o mecanismo mais apropriado para o controle de tráfego na Internet de hoje, em detrimento de muitos mecanismos de AQM (*Active Queue Management*).

Essa diversidade de opiniões sobre a aplicabilidade dos mecanismos de gerência de fila de espera leva a crer que cada mecanismo é caracterizado por apresentar melhor desempenho em dadas condições [de Castro et al., 2004]. Portanto, explorar essas especificidades, aplicando o melhor mecanismo para as condições de tráfego conhecidas pode trazer ganhos de desempenho.

O objetivo desse estudo de caso é analisar o comportamento de três configurações diferentes de gerência de fila de espera com relação ao tráfego agregado TCP e UDP, e em seguida modelar e testar um meta-comportamento que adapte um nó às condições conhecidas, aplicando a melhor escolha de mecanismo de gerência de fila de espera. O principal fator explorado nesse estudo de caso é a proporção de tráfego UDP sobre o tráfego agregado. Como fluxos UDP geralmente não têm como reagir às

notificações implícitas e explícitas de congestionamento e desencadear mecanismos de controle de fluxo da Internet, o efeito do tráfego UDP sobre as conexões TCP pode ser nocivo, podendo causar mesmo o fenômeno de “starvation” sobre as aplicações TCP.

5.1. Especificação do Cenário

Utiliza-se como base de experimentação um simulador de redes comercial *OPNET Modeler* [OPNET, 2004]. Sobre esse simulador, foi implementada a arquitetura descrita na seção anterior. A topologia testada é mostrada na Figura 3 e é composta de dois nós roteadores, um conjunto de servidores e cerca de 170 clientes. Dentre estes, 150 clientes utilizam constantemente um conjunto de serviços baseado no protocolo de transporte TCP e outros 20 clientes utilizam serviços baseados no protocolo UDP, porém variando-se o número de clientes UDP ativos ao longo do tempo. Os clientes e servidores estão localizados em extremidades opostas de um enlace de 20 Mbps que interliga os dois roteadores. Apesar da topologia simulada ser bastante simples, o comportamento das interfaces do roteador pode ser facilmente abstraído para qualquer interface de uma rede mais complexa.

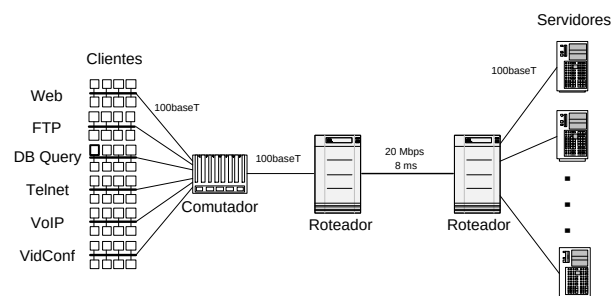


Figura 3: A Topologia Simulada.

5.2. Meta-Configurações

Os perfis de gerência de fila de espera escolhidos para análise são: FIFO com *Drop-Tail*, FIFO com RED e FIFO com RED habilitado com ECN (*Explicit Congestion Notification*) [Ramakrishnan et al., 2001]. Para facilitar a referência no restante do artigo, atribui-se as denominações *FIFO*, *RED* e *ECN*, respectivamente, para os perfis. Definiram-se então diversos cenários de *baseline* onde a proporção de tráfego UDP em relação ao tráfego TCP é variada. Foram testadas situações onde essa proporção varia de 5 a 85%. A partir de simulações sucessivas sobre o comportamento desses três perfis (*FIFO*, *RED* e *ECN*) sobre esses cenários, definiu-se uma meta-configuração que define o perfil a ser adotado para faixas de proporção de tráfego UDP sobre tráfego TCP (Figura 4).

Em seguida, foi definida uma maneira de determinar através de amostragem a proporção instantânea de tráfego UDP contida no tráfego agregado. Foi então escolhido coletar uma amostra (pacote) da interface a cada 4 ms. Na arquitetura implementada, essa coleta é realizada através da definição de um módulo programável do



Figura 4: Perfil Adotado de Acordo com Carga UDP.

tipo “Statistic”. O tipo do protocolo de nível de transporte para cada amostra é então coletado e armazenado em um vetor histórico de 1000 posições. Então, a cada 10 s, o meta-comportamento de avaliação é ativado, analisando a amostragem realizada nos últimos 4 s e tomando a decisão adequada prevista. A estimativa da proporção de tráfego UDP por amostragem parametrizada por esses valores se mostrou suficientemente precisa, como pode ser observado na Figura 5. Nesta Figura, observa-se que para um cenário misto, com proporção de tráfego UDP variável, o método por estimativa está bastante próximo do real que pode ser constatado por medidas precisas.

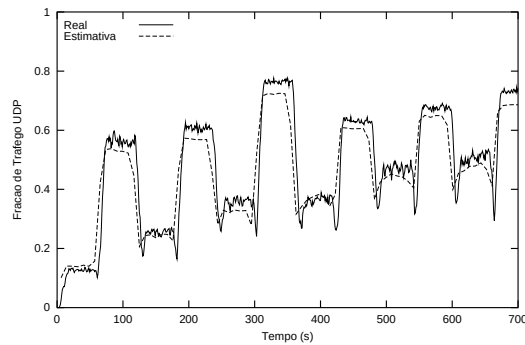


Figura 5: Estimativa da Carga UDP por Amostragem.

Definiram-se então duas experiências. Na primeira, o objetivo é de minimizar especificamente o tempo de carga de uma página no serviço *Web*, em detrimento a todos os outros serviços e parâmetros de desempenho de rede.

Para o segundo experimento, definiu-se 10 critérios, que reúnem indicadores de desempenho de rede (Taxa de perda de pacotes e Taxa média de ocupação da fila) e de aplicações TCP e UDP (HTTP, DB, FTP, Telnet, videoconferência e Voz sobre IP). Para cada um desses critérios foi atribuído um peso w , que representa a importância do critério no cálculo da função objetivo que se quer definir. Os critérios e seus respectivos pesos atribuídos são apresentados na Tabela 1.

Tabela 1: Atribuição dos Pesos aos Critérios.

Critério	HTTP	DB	FTP	Telnet	Perda	Ocup.	VC Atraso	VC Jitter	VoIP Atraso	VoIP Jitter
Peso	6.0	4.0	1.0	1.0	2.0	0.7	1.0	0.2	1.0	0.2

Para cada critério c , calcula-se o fator de normalização entre os desempenhos médios obtidos com a aplicação dos perfis *FIFO*, *RED* e *ECN*:

$$F_c = \text{Max} \left(\bar{P}_c^{FIFO}; \bar{P}_c^{RED}; \bar{P}_c^{ECN} \right)$$

Para cada perfil M , dá-se o ganho relativo como sendo:

$$G^M = \frac{1}{\sum w_c} \times \sum_{c=1}^C \left[w_c \times \left(1 - \frac{\bar{P}_c^M}{F_c} \right) \right] \quad (1)$$

onde w_c é o peso atribuído ao critério. O objetivo nesse segundo experimento é, então, maximizar a função objetivo definida pela Equação 1.

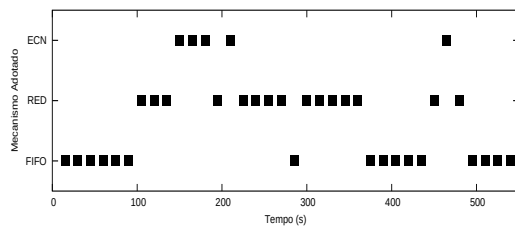
Após a simulação de diversas situações de tráfego UDP em separado para gerar as meta-configurações, criou-se então um cenário misto onde o número de clientes UDP é variado. Os elementos de rede não têm informação *a priori* dessas variações, cabendo ao mecanismo de detecção por amostragem detectar tais mudanças. Todas as simulações posteriores são então baseadas neste novo cenário misto. O objetivo é de testar as meta-configurações em situações de tráfego não utilizadas anteriormente.

6. Resultados e Discussão

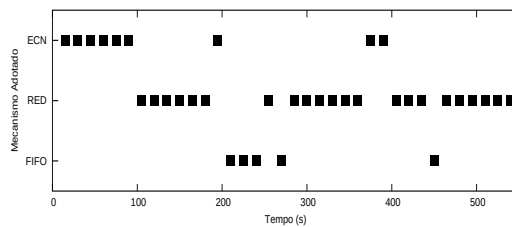
Para cada experimento, foi realizado um determinado número de simulações de replicações do cenário proposto (variando-se as sementes) de maneira que obtivesse, por exemplo, para o serviço HTTP, uma média $\mu = 0.7802s$ e um intervalo de confiança de $\pm 0.059s$ para $\alpha = 0.05$. A duração de cada simulação é de 670 s e os resultados analisados não levam em conta um período transiente inicial de 130 s, determinado através de técnica descrita em [Jain, 1991].

Comparou-se então o desempenho dos três perfis definidos e do perfil adaptativo (citado como *Active* nos resultados) sobre o cenário misto citado na Seção 5.2., onde a carga UDP varia de acordo com o tempo. O meta-comportamento definido em *Active* impôs aos dois experimentos a adaptação da gerência de fila de espera, resultando em comportamentos como os mostrados na Figura 6.

A Figura 7 mostra como exemplo a comparação entre o desempenho do serviço Web entre os quatro perfis testados. No primeiro experimento (Figura 7(a)), onde o objetivo é minimizar o tempo médio de carga de uma página Web, observa-se que o objetivo foi alcançado. No segundo experimento (Figura 7(b)), onde o objetivo é maximizar o valor da função objetivo definida pela Equação 1, observa-se que o desempenho do serviço Web para o perfil *Active* não foi o melhor entre todos os perfis testados. Entretanto, como a função objetivo definida leva em conta também o desempenho dos outros critérios, este resultado não é suficiente para afirmar que a função objetivo não foi maximizada.

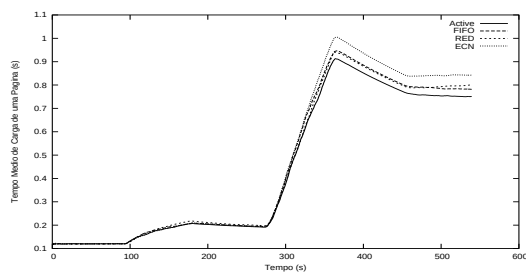


(a) Min. Tempo de Carga HTTP

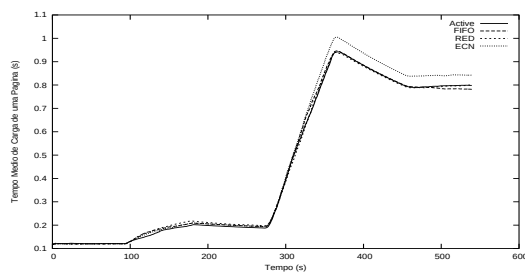


(b) Max. Função Objetivo

Figura 6: Comportamento da Gerência de Fila de Espera.



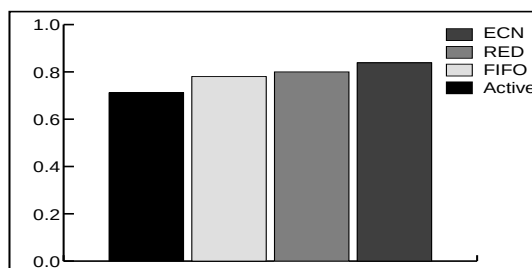
(a) Min. Tempo de Carga HTTP



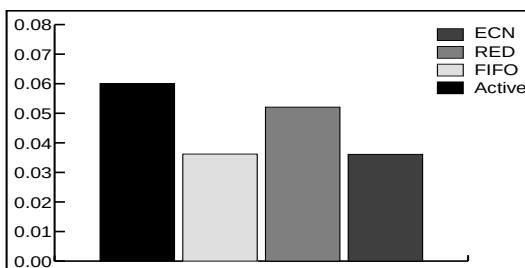
(b) Max. Função Objetivo

Figura 7: Comparação do Desempenho do Serviço Web nos Dois Objetivos Testados.

A Figura 8 mostra a comparação entre os desempenhos dos quatro perfis para os objetivos especificados nos dois experimentos. Na Figura 8(a), observa-se que o perfil *Active* é realmente o que apresenta menor média de tempo de carga de uma página Web, cumprindo o objetivo da meta-configuração. Na Figura 8(b), por sua vez, observa-se que o perfil *Active* é o que apresenta maior valor para a função objetivo definida na meta-configuração.



(a) Min. Tempo de Carga HTTP (s)



(b) Max. Função Objetivo

Figura 8: Comparação dos Resultados Obtidos.

7. Conclusões e Trabalhos Futuros

Este trabalho propõe uma abordagem baseada em Redes Ativas e Programáveis para a gerência adaptativa de QoS. Para isso, uma nova arquitetura de gerência adaptativa e sua linguagem de programação são apresentadas. Baseado nesta arquitetura, um estudo de caso envolvendo a gerência adaptativa de filas em roteadores foi realizado.

Os resultados obtidos mostram que é possível se obter melhorias no desempenho das aplicações de maneira geral apenas adaptando a configuração de gerência de modo a explorar os pontos fortes dos mecanismos disponíveis. O estudo de caso mostrou que dispondo apenas de mecanismos simples de gerência de filas de espera, como FIFO com *Drop-Tail*, FIFO com RED e FIFO com RED+ECN e da possibilidade de permutar esses mecanismos é possível se observar ganhos reais de desempenho.

É importante ressaltar que o *overhead* da inclusão dessas novas capacidades ativas não interfere no desempenho do plano responsável pelo encaminhamento dos pacotes (*forwarding*), uma vez que a arquitetura apresentada especifica a separação das funções de gerência e de encaminhamento de pacotes, baseado na proposta ForCES (*Forwarding and Control Elements Separation*) [Khosravi et al., 2003]. Portanto, o funcionamento de um plano não deve influir no desempenho do outro, a separação de poder computacional entre essas entidades.

O potencial dessa arquitetura, entretanto, vai além da simples manipulação de configuração de gerência de filas de espera. Vários outros módulos programáveis estão disponíveis, assim como a possibilidade de acesso a outros mecanismos de gerência de QoS. A exploração destas capacidades adicionais está prevista como trabalho futuro. Um estudo mais aprofundado sobre segurança, estabilidade e escalabilidade também se faz necessário. Pretende-se, em seguida, implementar a arquitetura proposta, através da criação de *software router* baseado em Linux, por exemplo.

Referências

- ANEP (2004). *The SwitchWare Project: ANEP (Active Network Encapsulation Protocol)*. <http://www.cis.upenn.edu/~switchware/ANEP/>.
- Blake, S., Black, D. L., Carlson, M., Davies, E., Wang, Z., and Weiss, W. (1998). An architecture for differentiated service. RFC 2475, Internet Engineering Task Force.
- Braden, B., Clark, D. D., Crowcroft, J., Davie, B. S., Deering, S. E., Estrin, D., Floyd, S., and Jacobson, V. (1998). Recommendations on queue management and congestion avoidance in the Internet. RFC 2309, Internet Engineering Task Force.
- Braden, R., Clark, D. D., and Shenker, S. (1994). Integrated services in the Internet architecture: an overview. RFC 1633, Internet Engineering Task Force.
- Christiansen, M., Jeffay, K., Ott, D., and Smith, F. D. (2001). Tuning RED for web traffic. *IEEE/ACM Transaction on Networking*, 9(3):249–64.

- de Castro, M. F., Merghem, L., Gaïti, D., and Mhamed, A. (2004). The basis for an adaptive IP QoS management. *IEICE Transactions on Communications - Special Issue Internet Technology IV*, E87-B(4):64–73.
- de Castro, M. F., M'hamed, A., and Gaïti, D. (2002). Providing core-generated feedback to adaptive media on actively managed networks. In *Proc. of IEEE/SBrT International Telecommunications Symposium (ITS 2002)*, Natal, Brazil.
- Floyd, S. and Jacobson, V. (1993). Random early detection gateways for congestion avoidance. *IEEE/ACM Transactions on Networking*, 1(4):397–413.
- Furini, M. and Towsley, D. F. (2001). Real-time traffic transmissions over the internet. *IEEE Trans. on Multimedia*, 3(1):33–40.
- Hong, D. P., Albuquerque, C., Oliveira, C., and Suda, T. (2001). Evaluating the impact of emerging streaming media applications on TCP/IP performance. *IEEE Communications Magazine*, pages 76–82.
- Hu, C.-L. and Chen, W.-S. (2000). A mobile agent-based active network architecture. In *Proc. of 7th International Conference on Parallel and Distributed Systems*, pages 445–452.
- Iannaccone, G., May, M., and Diot, C. (2001). Aggregate traffic performance with active queue management and drop from tail. *ACM SIGCOMM Computer Communication Review*, 31(3):4–13.
- ITU-T (1992). *BISDN Reference Model*. ITU-T Recommendation I.321.
- Jackson, A. W., Sterbenz, J. P. G., Condell, M. N., and Hain, R. R. (2002). Active network monitoring and control: The SENCOMM architecture and implementation. In *Proc. of DARPA Active Networks Conference and Exposition (DANCE'02)*.
- Jain, R. (1991). *The Art of Computer Systems Performance Analysis*. John Wiley & Sons.
- Khosravi, H., Anderson, T. W., and Eds. (2003). Requirements for separation of IP control and forwarding. RFC 3654, Internet Engineering Task Force.
- OPNET (2004). *OPNET Technologies Inc. Homepage*. <http://www.opnet.com>.
- Ramakrishnan, K. G., Floyd, S., and Black, D. L. (2001). The addition of explicit congestion notification (ECN) to IP. RFC 3168, Internet Engineering Task Force.
- Schwartz, B., Jackson, A. W., Strayer, W. T., Zhou, W., Rockwell, R. D., and Partridge, C. (2000). Smart Packets: Applying active networks to network management. *ACM Transactions on Computer Systems*, 18(1):67–88.
- Sterbenz, J. P. G. (2002). Intelligence in future broadband networks: Challenges and opportunities in high-speed active networking. In *Proc. of 2002 Intl. Zurich Seminar on Broadband Communications*, volume 2, pages 1–7.